Task 2: Data Analysis Report

Steven Kwan

Western Governors University

Data Science Capstone – D606

Daniel J. Smith, PhD, MBA

Introduction

Before starting this task, ensure that Task 1 has been approved by a course instructor. If Task 1 has not been approved, your Task 2 submission will be returned without evaluation.

In this task, you will address the real-world research question you identified in Task 1. You will gather relevant data, analyze the data, and recommend a data science solution. This recommendation will be included in a report that you will write, summarizing the key details of your analysis.

Note: You must successfully pass Task 1 before you begin working on Task 2. However, in practice, your topic, data scope, and research question may change based on your findings as you complete your research. Your work on Task 2 may lead you to recognize that more data is needed and that it must be approved data. For this reason, you may need to update your Task 1 forms before Task 2 is complete.

Note: Any information that would be considered confidential, proprietary, or personal in nature should not be included in the actual task submission to WGU. Do not include the actual names of people, suppliers, or companies, or other identifiable information. Fictional names should be used where necessary. Company-specific data, including financial information, should not be included but should be addressed in a general fashion, as appropriate.

Research Question

A. Summarize the real-data research question you identified in Task 1. Your summary should include justification for the research question you identified in Task 1, a description of the context in which the research question exists, and a discussion of your hypothesis.

Demand forecasting in the short-term rental market is essential for businesses seeking to optimize their pricing strategies and maximize revenue. At the same time, accurate demand forecasting helps guests make better decisions about whether to book a property immediately or delay booking due to wide availability. Traditional forecasting models usually look only at baseline property data, such as a listing's historical price, location, or day of the week. However, these internal metrics often miss sudden spikes in external demand caused by major concerts, sporting events, or festivals. This study tests whether integrating hyper-local live entertainment data from the Ticketmaster API significantly improves the forecasting accuracy of a Random Forest classifier.

Research Question: Using a random forest classifier, to what extent does integrating hyper-local event data from the Ticketmaster API improve the predictive accuracy of Seattle Airbnb long-term demand forecasting?

> **Null Hypothesis**- There is no statistically significant difference ($p > 0.05$) in predictive accuracy between the baseline control model and the event-enriched model, validated via a 5-fold cross-validation and a corrected resampled paired t-test ($\mu\Delta AUC \leq 0$). Any observed improvement is due to random chance.

Alternate Hypothesis- There is a statistically significant improvement ($p < 0.05$) in predictive accuracy when moving from the baseline control model to the event-enriched model, validated via a 5-fold cross-validation and a corrected resampled paired t-test ($\mu\Delta AUC > 0$).

It is important to note that 5-fold cross-validation violates the assumption of independence in the standard paired t-test, since the samples are reused after each fold. Therefore, applying the standard t-test would increase the chance of a Type 1 error, in which a statistically significant difference is falsely identified (Kiani, 2019). Nadeau and Bengio (2003) proposed a corrected, resampled paired t-test that accounts for data dependence and variance underestimation associated with cross-fold validation.

Data Collection

B. Report on your data-collection process by describing the relevant data you collected, discussing one advantage and one disadvantage of the data-gathering methodology you used, and discussing how you overcame any challenges you encountered during the process of collecting your data.

The data collection process involved obtaining open-source Airbnb listing data and Ticketmaster event data. The Airbnb listing data is easily accessible as downloadable CSV files at www.insideairbnb.com (Inside Airbnb, 2024) and contains regional archives by scraped date. The Ticketmaster event data is accessible via an API, and the documentation is available here: https://developer.ticketmaster.com/products-and-docs/apis/discovery-api/v2/ (Discovery API 2.0, 2026). Using Postman (a graphical user interface for testing APIs), I retrieved Seattle event data and saved the API response to a JSON file. Finally, I retrieved venue maximum capacities from Wikipedia (“Climate Pledge Arena," 2023; "Lumen Field," 2021; "T-Mobile Park," 2025).

One advantage of my data-gathering methodology is that it combines listing and event data from two different sources. This simulates real-world dynamics, as major events draw people from all over the world into the city where they are occurring. Technically speaking, having the listing data prepared in a CSV file and the event data in a JSON file made these simple to load and transform using the pandas library. In other words, I was not forced to scrape the data from the web tediously myself.

One disadvantage of my data-gathering methodology is the lack of historical flexibility – particularly because I could not retrieve historical event data from the Ticketmaster API. Since my Airbnb listing data was scraped on 6/15/2026 and my Ticketmaster event data was scraped on 8/1/2026, the lack of historical event data prevented me from analyzing short-term trends. I had to adapt by using a forward-looking date range of 8/1/2026 – 8/31/2026 for my analysis, which indicates a lead time of 46 – 77 days. Therefore, my model will train on and predict long-term booking demand using the date window 8/1/2026 – 8/31/2026.

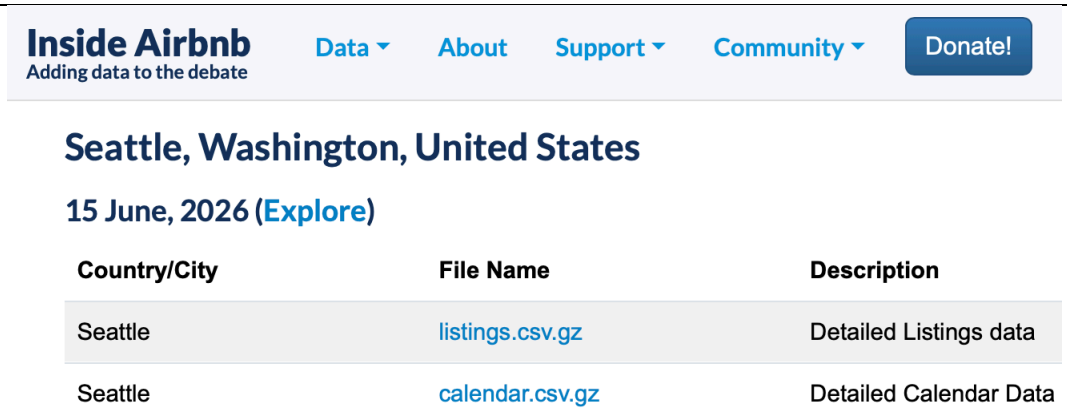
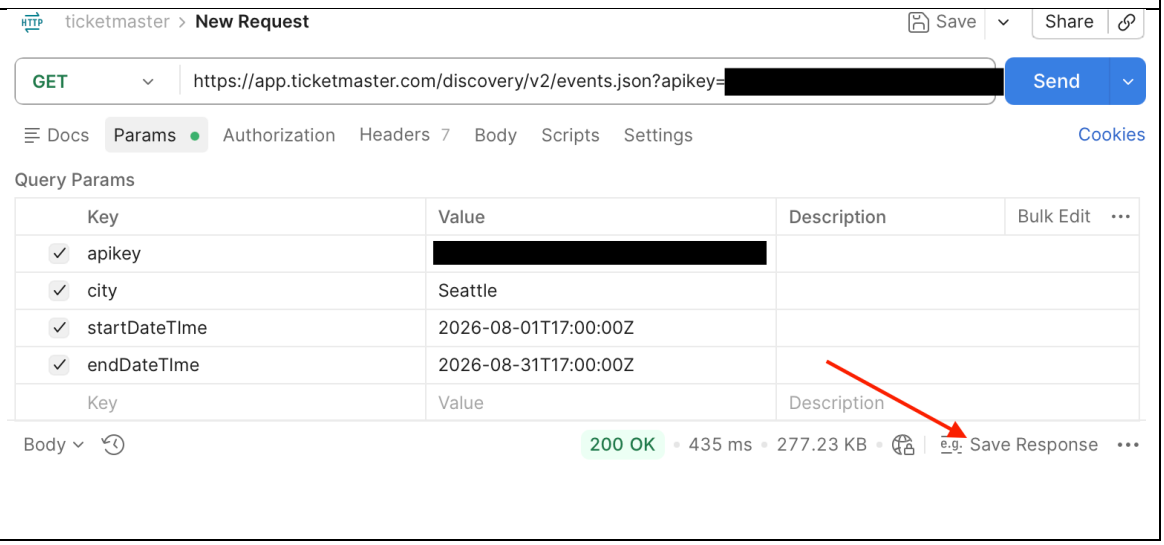
I encountered two major challenges during this process. First, being bound to the last scraped date (6/15/2026) was challenging, as it only provided a snapshot of booked and un-booked listings at that time (a forward-looking 12 months from that date). Furthermore, the "lead time" in the hospitality industry is an important metric, indicating the time between a customer's purchase (booking) and the date of stay (Katz et al., 2025). According to SiteMinder's Hotel Booking Trends, the average lead time in hotel bookings is 36 days (Elphick, 2024). This implies that training our model beyond 36 days from the scraped date may not provide sufficient short-term booking accuracy. I overcame this by accepting that my model would instead focus strictly on long-term demand forecasting, which also provides business value.

The second challenge was merging the listing and event data, which was somewhat difficult. Joining on the listing/event date appeared simple enough; however, if multiple events occur on a single date, the event features must be aggregated to maintain a 1:1 relationship with the listing date. I overcame this by aggregating the event features so that the listing date receives the appropriate weight for the events occurring that day.

Data Extraction and Preparation

C. Describe your data-extraction and data-preparation process and provide screenshots to illustrate *each* step. Explain the tools and techniques you used for data extraction and data preparation, including how these tools and techniques were used on the data. Justify why you used these particular tools and techniques, including one advantage and one disadvantage when they are used with your data-extraction and data-preparation methods.

Data Extraction (Describe your data-extraction process and provide screenshots to illustrate each step)

<table>
<tr>
<td>To extract Seattle Airbnb listing data, I downloaded the “listing” and “calendar” CSV files from Inside Airbnb (2024).</td>
<td> 
</td>
</tr>
<tr>
<td>To extract Seattle event data, I used Postman to make a GET request to the Discovery API (2026) with the correct query parameters for the city and date range. I then saved the response as a JSON file.</td>
<td> 
</td>
</tr>
</table>

To extract venue maximum capacity, I used Wikipedia (“Climate Pledge Arena," 2023; "Lumen Field," 2021; "T-Mobile Park," 2025). I then created a simple DataFrame with that information.	``` capacities = pd.DataFrame([{'venue_name': 'Climate Pledge Arena', 'max_capacity': 17151}, {'venue_name': 'Lumen Field', 'max_capacity': 68740}, {'venue_name': 'T-Mobile Park', 'max_capacity': 47368}]) ```

Data Preparation (Describe your data-preparation process and provide screenshots to illustrate each step)

1	Open listings.csv.	`listings = pd.read_csv('data/listings.csv')` `#listings.info(show_counts=True)`
2	Keep listings that have availabilities in the next 365 days (remove inactive listings).	`# Remove completely inactive listings` `listings = listings[listings['availability_365'] > 0]`

3	Keep baseline listing features.
	
	Drop NAs in the "price" column.
	
	Update price from string to float data type.
	
	Rename the neighborhood column.
		`# Filter listing features`
		`listings = listings[['id','neighbourhood_cleansed', 'price', 'latitude', 'longitude']]`
		
		`# Drop NAs in price`
		`listings.dropna(subset=['price'], inplace=True)`
		
		`# Update price from string to float64`
		`listings['price'] = listings['price'].str.replace('$', '').str.replace(',', '')`
		`listings['price'] = listings['price'].astype('float64')`
		
		`# Rename column`
		`listings.rename(columns={'neighbourhood_cleansed':'neighborhood'}, inplace=True)`

4	Open calendar.csv.
	
	Keep baseline calendar features.
	
	Encode target variable and update name to "is_booked."
	
	Drop the "available" column.
	
	Change "date" from string to a date-time data type.
	
	Slice date range (8/1 -8/31/2026).
	
	Add a column for the day of the week.
		`# Note: Calendar is forward-looking. Scraped date: 6/15/2026`
		`calendar = pd.read_csv('data/calendar.csv')`
		
		`# Select needed features`
		`calendar = calendar[['listing_id', 'date', 'available']]`
		
		`# Encode target variable`
		`# 0 = Vacant (Low Demand), 1 = Booked/Taken (High Demand)`
		`calendar['is_booked'] = calendar['available'].map({'t': 0, 'f': 1})`
		
		`# Drop "available" column`
		`calendar = calendar.drop(columns='available')`
		
		`# Convert date to datetime`
		`calendar['date'] = pd.to_datetime(calendar['date'])`
		
		`# Slice a 30 day window to ensure data integrity (sparse/inaccurate data if listing is too far out)`
		`calendar_30 = calendar[(calendar['date'] >= '2026-08-01') & (calendar['date'] <= '2026-08-31')].copy()`
		
		`# Add column for day of week`
		`calendar_30['day_of_week'] = calendar_30['date'].dt.day_name()`

5	Inner join the listing data with the calendar data on the listing_id.	
6	Load and parse JSON file containing Seattle event data. Initialize target columns. Create a “for” loop that iterates the events and appends values to the appropriate python list.	
7	Create a structured DataFrame for the Ticketmaster event data, “tm_df”.	
8	Change latitude and longitude to numeric float data types.	
9	Change “event_date” to a date-time data type.	

```python
# Join calendar_30 and listings

df = pd.merge(calendar_30, listings, left_on='listing_id', right_on='id', how='inner')
```

```python
# Load the file
with open('data/response.json', 'r') as file:
    data = json.load(file)

events_list = data.get('_embedded', {}).get('events', [])

# Initialize target columns
names = []
dates = []
venue_names = []
lats = []
longs = []

for event in events_list:
    # Event Core Data
    names.append(event.get('name', None))
    dates.append(event.get('dates', {}).get('start', {}).get('localDate', None))

    # Venue Details
    venues = event.get('_embedded', {}).get('venues', [])
    if isinstance(venues, list) and len(venues) > 0:
        v = venues[0]  # Grab the primary venue object cleanly
        venue_names.append(v.get('name', None))
        lats.append(v.get('location', {}).get('latitude', None))
        longs.append(v.get('location', {}).get('longitude', None))
    else:
        venue_names.append(None)
        lats.append(None)
        longs.append(None)
```

```python
# Build the final structured DataFrame
tm_df = pd.DataFrame({
    'event_name': names,
    'event_date': dates,
    'venue_name': venue_names,
    'venue_latitude': lats,
    'venue_longitude': longs
})
```

```python
# Convert coordinates into numeric float data types safely
tm_df['venue_latitude'] = pd.to_numeric(tm_df['venue_latitude'], errors='coerce')
tm_df['venue_longitude'] = pd.to_numeric(tm_df['venue_longitude'], errors='coerce')
```

```python
# Convert time to datetime
tm_df['event_date'] = pd.to_datetime(tm_df['event_date'], errors='coerce')
```

10	Create a DataFrame with each venue's max capacity.	`capacities = pd.DataFrame([ {'venue_name': 'Climate Pledge Arena', 'max_capacity': 17151}, {'venue_name': 'Lumen Field', 'max_capacity': 68740}, {'venue_name': 'T-Mobile Park', 'max_capacity': 47368} ])`
11	Left join the Ticketmaster event data with the venue capacity data on "venue_name."	`tm_df = pd.merge(tm_df, capacities, how='left', on='venue_name')`
12	Group events by date and aggregate venue data upstream. Get the global event count, sum of capacities, and create a "geo-payload" to be used for capacity-weighted distance calculations.	`# Upstream aggregation group def upstream_agg(group): return pd.Series({ 'global_event_count': len(group), 'total_daily_capacity': group['max_capacity'].sum(), # Pack the coordinate pairs and capacities into a list of tuples for step 2 'geo_payload': list(zip(group['venue_latitude'], group['venue_longitude'], group['max_capacity'])) }) df_events_aggregated = tm_df.groupby('event_date').apply(upstream_agg).reset_index()`
13	Left join the listing data with the Ticketmaster data on the listing date as "final_df". (Note that dates without events will have null values, which will be filled in subsequent steps.)	`# Join with Ticketmaster df on date final_df = pd.merge(df, df_events_aggregated, how='left', left_on='date', right_on='event_date')`

14	Define a function that returns the capacity-weighted distance in miles using the Haversine distance (2022), which measures the distance between two points (specified by their latitude and longitude) on the surface of a sphere.
	
	Capacity-weighted distance is the calculated by taking the sum of each distance multiplied by the respective proportion of total capacity.
		`from haversine import haversine, Unit`
		
		`def compute_weighted_distance(row):`
		`  payload = row['geo_payload']`
		
		`  # Safe check for valid payload`
		`  if not isinstance(payload, list) or len(payload) == 0:`
		`  return np.nan`
		
		`  # Origin point as a (lat, lon) tuple in degrees`
		`  origin = (row['latitude'], row['longitude'])`
		
		`  distances = []`
		`  capacities = []`
		
		`  # Calculate distances using the haversine package`
		`  for ev_lat, ev_lon, capacity in payload:`
		`  destination = (ev_lat, ev_lon)`
		
		`  # Computes distance in miles directly`
		`  dist = haversine(origin, destination, unit=Unit.MILES`
		
		`  distances.append(dist)`
		`  capacities.append(capacity)`
		
		`  # Convert to numpy arrays for calculation`
		`  distances = np.array(distances)`
		`  capacities = np.array(capacities)`
		`  sum_capacity = np.sum(capacities)`
		
		`  if sum_capacity > 0:`
		`  # Return capacity-weighted distance average`
		`  return np.sum(distances * capacities) / sum_capacity`
		`  else:`
		`  # Fallback: Return unweighted average distance`
		`  return np.mean(distances)`

15	Apply the function creating a capacity-weighted distance column.
	
	For dates without an associated event, fill the global event count and total daily capacity with 0.
	
	Drop the payload structure.
		`# Apply the distance calculation safely`
		`final_df['capacity_weighted_distance'] = final_df.apply(compute_weighted_distance, axis=1)`
		
		`# Fill Nans`
		`final_df['global_event_count'] = final_df['global_event_count'].fillna(0).astype(int)`
		`final_df['total_daily_capacity'] = final_df['total_daily_capacity'].fillna(0)`
		
		`# Drop payload structure safely (errors='ignore' ensures it won't crash if already dropped)`
		`final_df.drop(columns=['geo_payload'], errors='ignore', inplace=True)`

17	Remove duplicate or unnecessary columns in final DataFrame.	`# Remove unnecessary columns` `final_df = final_df.drop(columns=['latitude', 'longitude', 'event_date', 'id'])`
18	Save final DataFrame to csv file.	`# Save final_df to csv file` `final_df.to_csv('seattle_events_airbnb.csv', index=False)`

Tools and Techniques (Explain the tools and techniques you used for data extraction and data preparation, including how these tools and techniques were used on the data.) Justify why you used these particular tools and techniques, including one advantage and one disadvantage when they are used with your data-extraction and data-preparation methods.)

To extract the Airbnb data, I used a web browser to download the listings.csv and calendar.csv files from Inside Airbnb's (2024) Seattle archives (scraped on 6/15/2026). To extract Seattle event data, I used Postman to call the Ticketmaster Discover API, sending a GET request with the appropriate query parameters to retrieve events for 8/1 – 8/31/2026. I saved the API response to a JSON file. Finally, I used Wikipedia to extract each event venue's maximum capacity and created a custom pandas DataFrame to store that information.

To prepare my data, I created a Jupyter Notebook file within Visual Studio Code. I set up my environment using conda, which contained a baseline environment with dependencies and libraries already installed. Within the Jupyter Notebook file, I imported libraries and dependencies, including json, pandas, NumPy, and haversine. The data preparation and transformation steps relied heavily on the pandas library, which provided the necessary tools for data pre-processing. The json library was used to load and parse the JSON file. The NumPy and haversine libraries were used to calculate the capacity-weighted distance.

Using the pandas library, basic techniques such as filtering, dropping columns, updating data types, inspecting, and joining DataFrames were used. One of the more challenging techniques used during data preparation involved computing the capacity-weighted distance (defined as the distance between the listing and one or more venues, weighted by each venue's maximum capacity). To obtain the capacity-weighted distance, I first aggregated Seattle event data by event date, creating a payload containing each venue's latitude, longitude, and maximum capacity. By aggregating the event data by date, I maintained a 1:1 relationship between each listing's calendar date and its event data. After joining the listing and event data, I now had access to each listing's latitude and longitude, as well as the event payload. I applied the Haversine formula to calculate the distance from each venue to the listing. Then I obtained the capacity-weighted distance by summing each distance multiplied by its respective proportion of total capacity. Note that the Haversine distance (2022) measures the distance between two points (specified by their latitude and longitude) on the surface of a sphere. After obtaining the capacity-weighted distance for each row, I removed unnecessary columns and saved my final DataFrame as a CSV file for later use.

Justification of Tools and Techniques (Justify why you used these particular tools and techniques, including one advantage and one disadvantage when they are used with your data-extraction and data-preparation methods.)

For the data preparation phase, a Jupyter Notebook in Visual Studio Code was paired with a conda environment to write data pre-processing scripts. This environment offered flexibility and convenience when editing and executing code, while simplifying the process of importing and installing the required libraries. As previously mentioned, libraries such as json, pandas, NumPy, and haversine were used to pre-process the data programmatically.

Especially useful was the pandas library, as it helped transform and inspect the data. While vanilla Python or other data analysis packages could achieve the same result, using pandas was efficient given my familiarity with the library and its powerful tools for transforming data.

Most of the techniques used during data preparation were basic transformation steps, such as filtering, joining, inspecting, dropping columns, and simple arithmetic operations. One of the more complicated techniques employed was creating a payload containing venues' latitude, longitude, and capacity data. Initially, I did not consider the need for an upstream payload. However, it became clear after inspecting my dataset that listing dates were being duplicated when joining the event data by the event date. Aggregating event data into a payload before joining proved to be a much cleaner methodology than trying to aggregate the same data downstream.

One Advantage of Tools and Techniques

One advantage of my data extraction process was the wide accessibility of the data. The tools and techniques I employed were not difficult to implement since most of my data could be downloaded as CSV files or manually extracted from Wikipedia. Calling the Ticketmaster Discovery API using Postman proved slightly more difficult; however, it remained relatively simple to obtain this data given the succinct API documentation. An advantage of my data preparation process was the use of powerful libraries to transform my data. Without pandas and NumPy, I could not have easily transformed my data into a clean data set. As previously mentioned, I needed to create a payload of aggregated event data to avoid duplication of rows when joining. Without the help of the pandas library, this would have been much more challenging.

One Disadvantage of Tools and Techniques

One disadvantage of my data extraction process was that I could not obtain the exact date ranges that I had originally intended to use. For example, to predict short-term listing availability with a lead time of less than 30 days, I needed to obtain historical event data starting on 6/15/2026 (the listing scraped date). However, historical event data was not available for querying via the Ticketmaster Discovery API. Instead of using the API, I could have scraped the web for Seattle events that occurred in June. In the data preparation phase, one disadvantage was the sheer amount of preprocessing I needed to perform, since the data sets were coming from several different sources. Instead of combining several tables, I could have searched for datasets on Kaggle where similar data may already have been compiled.

Analysis

D. Report on your data analysis process by describing the analysis technique(s) you used to appropriately analyze the data. Include the calculations you performed and their outputs. Justify how you selected the analysis technique(s) you used, including one advantage and one disadvantage of *each* technique.

Data Analysis Process and Techniques

I. Environment Setup and Final Data Inspection

To ensure reproducibility of my analysis, I set a fixed seed immediately after importing the necessary libraries into a new Jupyter Notebook file. Upon inspection of the saved data set, there were still NaN (not a number) values in the capacity-weighted distance column. Since these NaN values corresponded to dates without an event, I filled them with 9999 to indicate a large distance from the listing. **Justification**: While this is not a perfect solution, the less favorable options were to remove the 75,000+ samples containing NaN or set the capacity-weighted distance to 0. Removing 75,000+ rows would severely impact model training, as they represent about 35% of my final dataset. Alternatively, using a distance of 0 implies that the listing is inside the venue, which would confuse the model.

Turning my attention to the day_of_week variable, I created a new variable to indicate whether the day is a weekend day. I expect this change to boost my model's performance by avoiding training on a one-hot encoded sparse matrix (a table filled with mostly zeros). **Justification**: An indicator of whether the day is a weekend provides a more robust (stronger) signal, as bookings are more likely to occur on weekends.

Finally, I included another important feature, the lead time, which I calculated by subtracting the listing date from the scraped date. **Justification**: As previously explained, lead time is an important metric in the hospitality industry, given that it is a strong indicator of demand.

One Advantage: Performing a final data inspection, filling NaN values, and ensuring feature reliability provided the model with clean training data. Neglecting to perform a final data inspection may lead to downstream problems if values are missing or do not make sense to the model. In this case, one of the more difficult decisions was how to fill NaN values in the capacity-weighted distance column. The advantage of filling the NaNs with a high value is that it signals to the model that the event's venue is far away and, therefore, less likely to be booked based on event variables. Eliminating 75,000 rows of data was likely the worst choice, as it would remove all listings that did not correspond to an event. This would result in an inaccurate, biased model since listings without events were not included in training.

One Disadvantage: While there are few disadvantages to performing a final inspection, my decision to encode the day_of_week feature as a binary variable (is_weekend) may eliminate some important patterns in bookings. For example, Fridays and Mondays could be seen as popular days for bookings; however, only Saturdays and Sundays are counted as weekend days. Fine-tuning this feature in the future could be worthwhile to produce a better predictive model.

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import GroupKFold, cross_validate
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupShuffleSplit

from scipy import stats
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix
from sklearn.compose import ColumnTransformer

from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GroupKFold, RandomizedSearchCV

from category_encoders import TargetEncoder
from scipy.stats import randint
import random
```

```
# Set everything to a fixed seed
random.seed(42)
np.random.seed(42)
```

```
# Open csv
df = pd.read_csv('seattle_events_airbnb.csv')

df.info()
```

✓ 0.0s

```
<class 'pandas.DataFrame'>
RangeIndex: 215047 entries, 0 to 215046
Data columns (total 9 columns):
 #   Column                      Non-Null Count   Dtype
---  ------                      --------------   -----
 0   listing_id                  215047 non-null  int64
 1   date                        215047 non-null  str
 2   is_booked                   215047 non-null  int64
 3   day_of_week                 215047 non-null  str
 4   neighborhood                215047 non-null  str
 5   price                       215047 non-null  float64
 6   global_event_count          215047 non-null  int64
 7   total_daily_capacity        215047 non-null  float64
 8   capacity_weighted_distance  124866 non-null  float64
dtypes: float64(3), int64(3), str(3)
memory usage: 14.8 MB
```

```
# Fill NaNs
df['capacity_weighted_distance'] = df['capacity_weighted_distance'].fillna(9999.0)
```

```
# Check if the day is in the list and convert True/False to 1/0
df['is_weekend'] = df['day_of_week'].isin(['Saturday', 'Sunday']).astype(int)
```

```
# Calculate lead time from scraped date

# Update data type of 'date' column
df['date'] = pd.to_datetime(df['date'])

# Date data was scraped
scraped_date = pd.to_datetime('2026-06-15')

# Create column for lead time
df['lead_time'] = (df['date'] - scraped_date).dt.days
```

II. Descriptive Statistics

Examining the descriptive statistics of the data set, such as the mean, max, and standard deviation, I observed that the maximum listing price exceeded $25,000. Plotting several histograms to view data distribution, the price variable exhibited extreme right-skew (see Exploratory Data Analysis in the next section). To handle extreme outliers and avoid data loss (Sugavasilakshmisahithi, 2025), I applied Winsorization by capping prices at the 99th percentile. **Justification**: The price of an Airbnb at the 99th percentile in this dataset was $1,478. While this is high, it is not extreme (as with a $25,000 listing). For example, a luxury Airbnb with 10 bedrooms and 8 bathrooms could be justified at $ 1,478 per night. When capped at this price, data loss is avoided, and the model still understands that this value is at the high end for pricing.

One Advantage: Analyzing descriptive statistics provides a clear understanding of the data's central tendency (the middle values) and dispersion (how spread out the numbers are). It also helps identify extreme outliers, which are often signaled when the mean of a variable is far below its maximum. In this dataset, the price variable had a mean of approximately $350 but a maximum exceeding $25,000. Reviewing these statistics allows researchers to spot potential data issues early and take corrective action. By capping the values, the Random Forest model can still interpret these samples as high-tier prices without being distorted by extreme outliers.

One Disadvantage: While there are few disadvantages to analyzing descriptive statistics, a potential disadvantage of capping prices at the 99th percentile is that the outliers could be meaningful. For example, in a real-world scenario, there could be rare instances of extreme prices, such as $26,000 per night for ultra-luxury listings. If these listings are booked at a higher rate than expected, capping the price could interfere with appropriate model training. This highlights the need to consider when outliers should be retained and when they should be modified.

```
# Descriptive statistics
df.describe()
```

	listing_id	date	is_booked	price	global_event_count	total_daily_capacity	capacity_weighted_distance
count	2.150470e+05	215047	215047.000000	215047.000000	215047.000000	215047.000000	215047.000000
mean	8.270289e+17	2026-08-16 00:00:00	0.396583	349.725273	0.645161	18696.548387	4195.200139
min	6.606000e+03	2026-08-01 00:00:00	0.000000	10.180000	0.000000	0.000000	0.052130
25%	4.790801e+07	2026-08-08 00:00:00	0.000000	171.500000	0.000000	0.000000	2.937102
50%	9.327041e+17	2026-08-16 00:00:00	0.000000	280.740000	1.000000	17151.000000	5.962833
75%	1.444585e+18	2026-08-24 00:00:00	1.000000	422.240000	1.000000	17151.000000	9999.000000
max	1.707715e+18	2026-08-31 00:00:00	1.000000	25916.670000	2.000000	85891.000000	9999.000000
std	6.495004e+17	NaN	0.489189	465.971871	0.598299	24562.799606	4932.293161

```python
# Calculate the 99th percentile and cap the values (Winsorization) to avoid data loss

# Calculate the price at the 99th percentile
price_99th = df['price'].quantile(0.99)
print(f"The 99th percentile price is: ${price_99th:.2f}")

# Cap the outliers (any price above the 99th percentile gets set to the 99th percentile value)
df['price'] = df['price'].clip(upper=price_99th)
```

```
The 99th percentile price is: $1478.00
The new maximum price in the dataset is: $1478.00
```

III. Exploratory Data Analysis (Univariate and Bivariate)

Univariate visualizations were used to help understand the distribution of individual variables. A count plot of the top 10 neighborhoods by listing volume revealed that the downtown Seattle neighborhoods of Broadway and Belltown had the highest listing volume. Also, a histogram grid was created to view data distributions of the remaining numeric variables. Finally, the target variable (is_booked) was examined to determine the percentages of booked (39.66%) and un-booked (60.34%) listings. **Justification**: Univariate visualizations provide essential baseline information about individual variables. A count plot of the categorical neighborhood variable was generated to evaluate the concentration of listings by neighborhood. For example, if 90% of listings were concentrated in the same neighborhood, it could indicate a biased sample. Additionally, a histogram grid is an efficient way to visualize the shape, spread, and skewness of multiple numeric variables at once. Finally, viewing the percentage of booked versus un-booked listings was important for assessing target class imbalance. For example, if 99% were labeled "un-booked," the model might recognize this surface-level pattern, predict "un-booked" for each test sample, and be correct 99% of the time. A correlation heatmap (**multivariate/bivariate visualization**) was created to assess positive, negative, or neutral relationships among variables. **Justification**: A correlation heatmap is an efficient way to assess relationships among variable pairs. This is used to gain a better understanding of how variables are interacting with one another.

One Advantage: Using univariate and bivariate visualizations helps uncover potential issues in the data, such as skewness, data concentration, and multicollinearity (when two variables are highly correlated). In the case of a severe class imbalance in the target variable, there could be significant bias towards predicting the majority class. Additionally, bivariate visualizations such as a correlation heatmap help identify which features have the strongest relationships with the target variable, thereby guiding better feature selection.

One Disadvantage: While there are few disadvantages to visualizing data, one potential issue is that hidden relationships may not be apparent in the correlation heatmap. The correlation heatmap shows linear relationships; however, if non-linear relationships are present, this would not be so obvious.

```
# Neighborhoods by listing volume

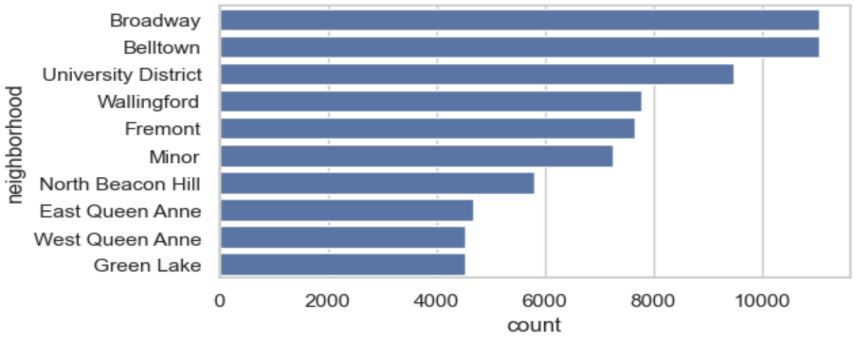
top_10_neighborhoods = df['neighborhood'].value_counts().head(10).reset_index()

plt.figure(figsize=(6, 3))

sns.barplot(
    data=top_10_neighborhoods,
    x='count',
    y='neighborhood',
)

plt.show()
```

Pyth



```python
# Histogram grid

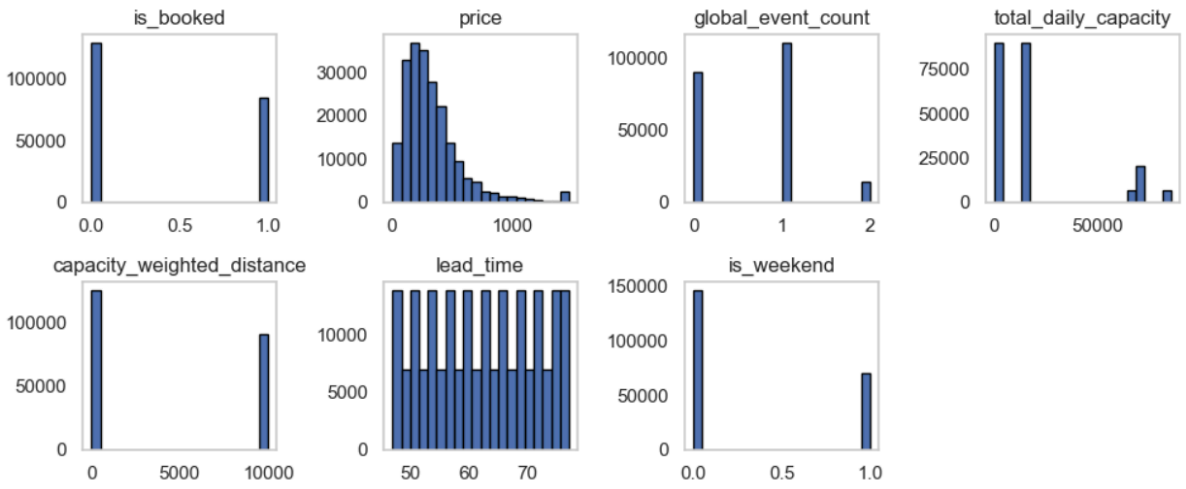
# Apply Seaborn's clean styling to the pandas plots
sns.set_theme(style="whitegrid")

# Plot features in grid
df.drop(columns=['listing_id', 'date']).hist(  # drop unnecessary features
    bins=20,
    figsize=(10, 6),
    layout=(3, 4),  # layout grid shape: 3 rows by 4 columns (adjust based on feature
    grid=False,
    edgecolor='black'
)

plt.tight_layout()

plt.show()
```

Python




```python
# Percent booked

count_booked = len(df[df['is_booked'] == 1])
count_available = len(df[df['is_booked'] == 0])

percent_booked = count_booked / (count_booked + count_available)
percent_available = count_available / (count_booked + count_available)

print(f"Percent booked: {(percent_booked * 100):.2f}%")
print(f"Percent available: {(percent_available * 100):.2f}%")
```

```
Percent booked: 39.66%
Percent available: 60.34%
```

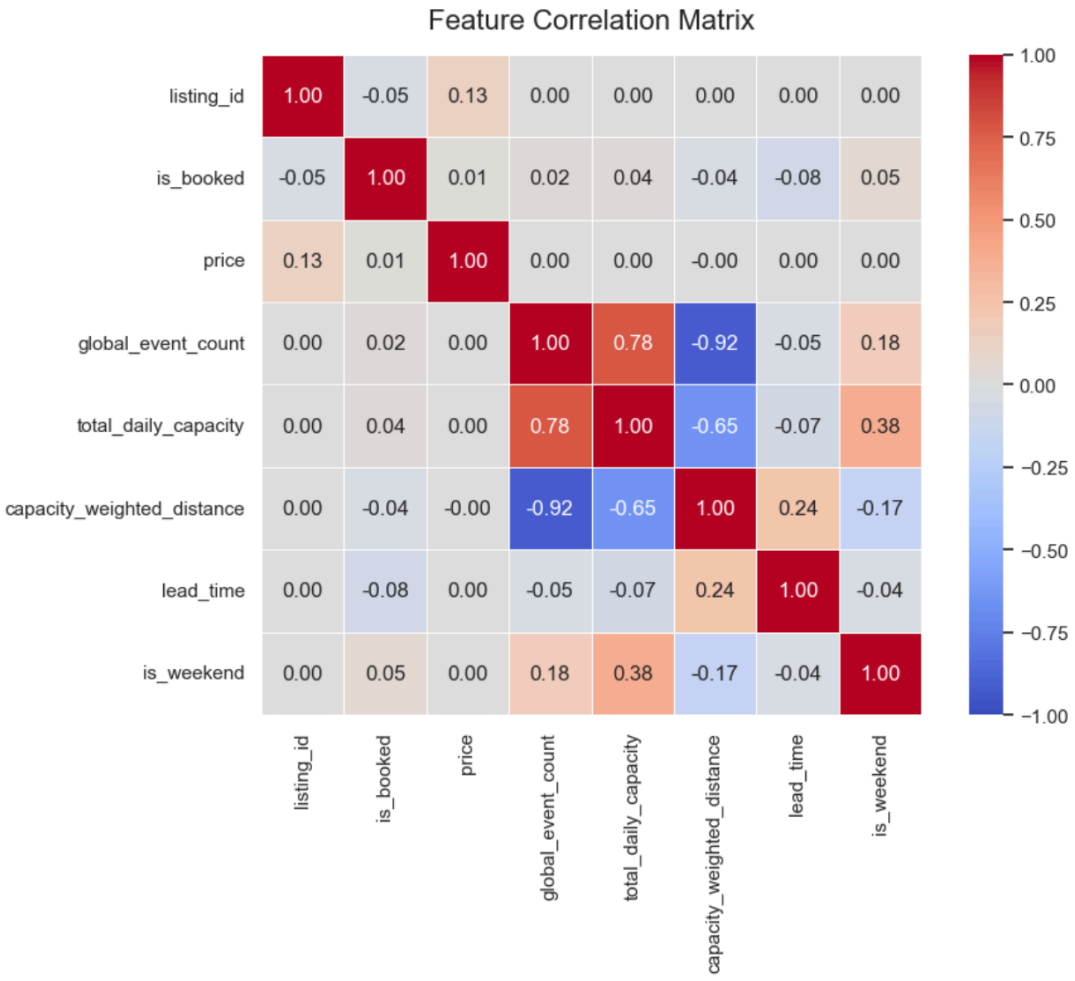
```python
# Correlation matrix

corr_matrix = df.corr(numeric_only=True)

plt.figure(figsize=(10, 8))

sns.heatmap(
    corr_matrix,
    annot=True,             # Show numeric correlation values inside the blocks
    fmt=".2f",              # Round numeric values to 2 decimal places
    cmap="coolwarm",        # Blue = Negative correlation, Red = Positive correlation
    vmin=-1, vmax=1,        # Set absolute min/max limits for the color bar scale
    linewidths=0.5,         # Add grid lines between the squares
    square=True             # Force grid blocks to be perfect squares
)

plt.title('Feature Correlation Matrix', fontsize=16, pad=15)
plt.tight_layout()
plt.show()
```



IV. Random Forest Classification Model

To begin training and cross-validation, I implemented the GroupShuffleSplit function from the scikit-learn library, which partitioned the dataset into training (80%) and test (20%) sets grouped by listing_id. **Justification**: The GroupShuffleSplit function is ideal in this case to prevent data leakage. If the data were split randomly without regard to the listing_id, the model could "cheat" during evaluation. This is because rows from the same listing would appear in both the training and testing phases, causing the model to memorize specific listing traits rather than generalizing well to new data.

After partitioning the data, I used my training set to create two distinct feature sets. The baseline feature set contained only listing variables, including price, is_weekend, neighborhood, and lead_time. The enriched feature set included the baseline features and the event features, including global_event_count, total_daily_capacity, and capacity_weighted_distance.

The neighborhood variable was intentionally left unencoded prior to the dataset partition. **Justification**: This approach prevents data leakage, ensuring that any subsequent encoding rules are learned exclusively from the training set. Scikit-learn's ColumnTransformer was used to transform the neighborhood column while leaving the remaining columns untouched.

The ColumnTransformer was implemented as the first step in a scikit-learn Pipeline, followed by model training with a Random Forest classifier. **Justification**: The Random Forest classifier was selected because this is a binary classification task (predicting one of two outcomes: booked versus un-booked). This model operates as a non-parametric ensemble method - meaning a normal Gaussian distribution is not assumed, which combines the predictions of multiple individual decision trees to improve overall accuracy and control over-fitting (Scikit-learn, n.d.).

Hyperparameter tuning/optimization was conducted via RandomizedSearchCV using 5-Fold GroupKFold cross-validation. **Justification**: This random search approach ensured that both the baseline and enriched feature sets were evaluated under identical search spaces and parameter ranges, granting them an equal opportunity for optimization.

Following the search, the optimal hyperparameter configurations were identified for each model. Finally, the Area Under the Receiver Operating Characteristic Curve (ROC-AUC) scores were extracted for each of the five folds to compare the cross-validation performance between the baseline and enriched models. **Justification**: The AUC score was selected because it evaluates the model's ability to correctly distinguish between the two classes (booked vs. un-booked) across all possible classification thresholds (Google, n.d.). Unlike standard accuracy, which only looks at a single fixed cutoff point and can be misleading even with mild class imbalances, ROC-AUC measures the balance between the true positive rate (sensitivity/recall) and the false positive rate (Google, n.d.). This provides a much more robust and comprehensive assessment of overall classification performance when comparing the baseline and enriched models.

One Advantage: An advantage of this training and validation pipeline is its high level of integrity regarding data leakage. By using scikit-learn Pipelines, GroupShuffleSplit, and GroupKFold, I ensured that information from the test sets and specific listing groups never leaked into the training process. Additionally, using RandomizedSearchCV is a highly efficient way to explore a wide range of hyperparameters without the massive computational expense of testing every single combination, allowing me to find an optimal model quickly. Using a Random Forest classification model fits perfectly

with my data set since it operates as a non-parametric ensemble method - meaning a normal Gaussian distribution is not assumed, which combines the predictions of multiple individual decision trees to improve overall accuracy and control over-fitting (Scikit-learn, n.d.).

One Disadvantage: A disadvantage of this approach is the high computational complexity and processing time. Training an ensemble method like Random Forest multiple times across 5 folds while simultaneously searching over random hyperparameters requires significant computational power. Additionally, while grouping by listing_id prevents data leakage, it might restrict the model's ability to learn if a specific subset of geographic locations (neighborhoods) becomes entirely tied to a single training fold.

```
# Initialize the group splitter
gss = GroupShuffleSplit(n_splits=1, train_size=0.8, random_state=42)

# Get the indices for the split based on the group
train_idx, test_idx = next(gss.split(df, groups=df['listing_id']))

# Create your dataframes
train_df = df.iloc[train_idx].reset_index(drop=True)
test_df = df.iloc[test_idx].reset_index(drop=True)
```

```
# Define features and target array
X_baseline = train_df.drop(columns=['listing_id', 'date', 'day_of_week', 'is_booked', 'global_event_count',
X_enriched = train_df.drop(columns=['listing_id', 'date', 'day_of_week', 'is_booked'])
y = train_df['is_booked']
groups = train_df['listing_id']

# Build the column transformer
column_transformer = ColumnTransformer(
    transformers=[
        ('neighborhood', TargetEncoder(smoothing=10.0), ['neighborhood'])
    ],
    remainder='passthrough'
)

# Build the base pipeline
pipeline = Pipeline(
    steps=[
        ('column_transformer', column_transformer),
        ('classifier', RandomForestClassifier(
            random_state=42,
            n_jobs=-1,
            class_weight='balanced_subsample'
        ))
    ]
).set_output(transform='pandas')

# Initialize GroupKFold
group_kfold = GroupKFold(n_splits=5)
```

```python
# Define distribution ranges for random sampling
param_distributions = {
    # Dynamically picks any integer from 5 up to 25
    'classifier__max_depth': randint(5, 26),
    # Dynamically picks any integer from 100 up to 500
    'classifier__n_estimators': randint(100, 501),

    # Hyperprams to control overfitting
    'classifier__min_samples_split': randint(2, 20),
    'classifier__min_samples_leaf': randint(1, 20),

    # Test different feature sub-selection strategies
    'classifier__max_features': [0.2, 0.4, 0.6, 0.8, 'sqrt', 'log2']
}
```

```python
# Tuning Baseline Model via Random Search
print("Running Randomized Search on Baseline Model...")
random_search_baseline = RandomizedSearchCV(
    estimator=pipeline,
    param_distributions=param_distributions,
    n_iter=5,                  # Evaluates 5 random combinations per model
    cv=group_kfold,            # Restricts cross-validation by listing groups
    scoring='roc_auc',         # Optimizes for Area Under the ROC Curve
    n_jobs=-1,                 # Uses all CPU cores
    random_state=42,           # Makes the random choices reproducible
    verbose=1
)
random_search_baseline.fit(X_baseline, y, groups=groups)

# Tuning Enriched Model via Random Search
print("\nRunning Randomized Search on Enriched Model...")
random_search_enriched = RandomizedSearchCV(
    estimator=pipeline,
    param_distributions=param_distributions,
    n_iter=5,
    cv=group_kfold,
    scoring='roc_auc',
    n_jobs=-1,
    random_state=42,
    verbose=1
)
random_search_enriched.fit(X_enriched, y, groups=groups)
```

```python
# Print Best Hyperparams and Best Out-of-Fold Scores
print("\n=== OPTIMIZED RESULTS ===")
print(f"Baseline - Best Params: {random_search_baseline.best_params_}")
print(f"Baseline - Best CV ROC AUC: {random_search_baseline.best_score_:.4f}")

print(f"\nEnriched - Best Params: {random_search_enriched.best_params_}")
print(f"Enriched - Best CV ROC AUC: {random_search_enriched.best_score_:.4f}")


# Function to print fold scores for a search object
def print_best_fold_scores(search_obj, name):
    best_idx = search_obj.best_index_
    print(f"\n{name} - Score per fold:")
    for i in range(5):
        score = search_obj.cv_results_[f'split{i}_test_score'][best_idx]
        print(f"  Fold {i+1}: {score:.4f}")

print_best_fold_scores(random_search_baseline, "Baseline")
print_best_fold_scores(random_search_enriched, "Enriched")
```

```
Running Randomized Search on Baseline Model...
Fitting 5 folds for each of 5 candidates, totalling 25 fits

Running Randomized Search on Enriched Model...
Fitting 5 folds for each of 5 candidates, totalling 25 fits

=== OPTIMIZED RESULTS ===
Baseline - Best Params: {'classifier__max_depth': 25, 'classifier__ma
Baseline - Best CV ROC AUC: 0.5946

Enriched - Best Params: {'classifier__max_depth': 25, 'classifier__ma
Enriched - Best CV ROC AUC: 0.6020

Baseline - Score per fold:
  Fold 1: 0.5973
  Fold 2: 0.5913
  Fold 3: 0.6024
  Fold 4: 0.5878
  Fold 5: 0.5940

Enriched - Score per fold:
  Fold 1: 0.6052
  Fold 2: 0.6003
  Fold 3: 0.6067
  Fold 4: 0.5964
  Fold 5: 0.6017
```

V. Corrected Resampled Paired T-test

Implementing a corrected, resampled paired t-test (Nadeau & Bengio, 2003) to assess statistical significance involved subtracting the baseline scores from the enriched scores. Using a Python implementation (Kateryna, 2021) of this test, the t-statistic was calculated to be 6.0497 with a corresponding p-value of 0.00377. Because the p-value is less than 0.05, this confirms that the addition of event features (the enriched feature set) had a statistically significant impact on the model's predictive performance. **Justification**: It must be noted that 5-fold cross-validation violates the assumption of independence in the standard paired t-test because the samples are reused after each fold. Therefore, using the standard t-test would increase the chance of a Type 1 error, in which a statistically significant difference is falsely identified (Kiani, 2019). Nadeau and Bengio (2003) proposed a corrected, resampled paired t-test that accounts for data dependence and variance underestimation associated with cross-fold validation.

One Advantage: One advantage to implementing a corrected resampled paired t-test is that it accounts for sample dependence due to cross-fold validation. It also minimizes the risk of a Type 1 error. Therefore, the results are more reliable than those from a standard t-test, in which the assumption of independence among samples would be violated.

One Disadvantage: While there are few disadvantages to implementing a corrected resampled t-test, it should be noted that standard Python libraries do not have native support for this test. Therefore, it is often coded manually, which can introduce errors if not implemented correctly.

```
baseline_cv_aucs = np.array(AUC_baseline)
enriched_cv_aucs = np.array(AUC_enriched)

# Compute differences and basic dimensions
differences = enriched_cv_aucs - baseline_cv_aucs
n_folds = len(differences)
mean_diff = np.mean(differences)
variance_diff = np.var(differences, ddof=1)
```

```
# Approach B: Nadeau & Bengio Corrected Resampled t-Test
# https://drogaieva.medium.com/designing-machine-learning-experiment-and-inte

# For 5-fold CV: test set = 20%, training set = 80%
ratio = 0.20 / 0.80
corrected_variance = variance_diff * ((1 / n_folds) + ratio)
corrected_t_stat = mean_diff / np.sqrt(corrected_variance)
# Two-tailed p-value with df = n - 1
corrected_p_val = stats.t.sf(np.abs(corrected_t_stat), df=n_folds-1) * 2
```

```
print("--- APPROACH B: Corrected Resampled t-test ---")
print(f"t-statistic: {corrected_t_stat:.4f}")
print(f"p-value:     {corrected_p_val:.5f}")
print(f"Significant? {corrected_p_val < 0.05}")
```

```
--- APPROACH B: Corrected Resampled t-test ---
t-statistic: 6.0497
p-value:     0.00377
Significant? True
```

VI. Evaluation of Hold-Out Test Set

To evaluate how well the optimized models generalize to unseen data, I applied them to the hold-out test set. I calculated the Area Under the Curve (AUC) scores for both the baseline and enriched models. The baseline model achieved an AUC of approximately 0.59, while the enriched model reached 0.61. Although a 2% improvement may seem small, it represents a measurable increase in predictive power.
Justification: A final evaluation on a hold-out dataset is crucial to verify that a model hasn't simply memorized the training data (overfitted) but can perform well in the real world. This step validates the entire modeling process and helps determine if further hyperparameter tuning or model selection is necessary.

One Advantage: Evaluating the hold-out test set provides a final performance metric, allowing us to analyze the model's true capabilities on unseen data.

One Disadvantage: There are few disadvantages to evaluating a hold-out test set; however, one might argue that the large amount of data (over 200,000 rows) could have allowed for a smaller hold-out test set (a larger training set). Allocating more data to the training set might have given the models the extra information they needed to boost performance.

```python
# Prepare hold-out features and true labels
# Ensure columns match training sets exactly
X_test_baseline = test_df.drop(columns=['listing_id', 'date', 'day_of_week', 'is_booked',
X_test_enriched = test_df.drop(columns=['listing_id', 'date', 'day_of_week', 'is_booked'])

y_test = test_df['is_booked']

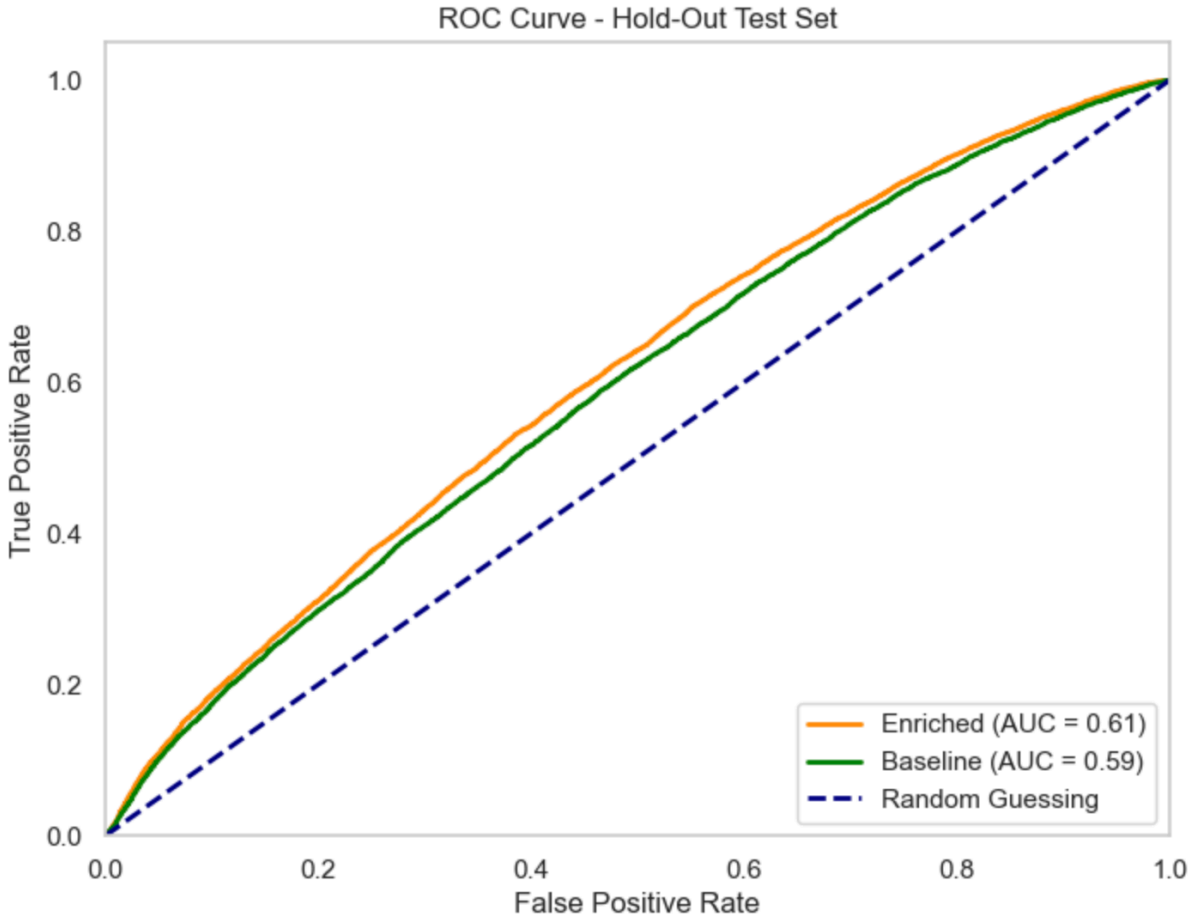
# Predict probabilities for the positive class (column index 1)
# Using random_search_baseline.predict_proba calls the best estimator automatically
proba_baseline = random_search_baseline.predict_proba(X_test_baseline)[:, 1]
proba_enriched = random_search_enriched.predict_proba(X_test_enriched)[:, 1]

# Calculate Hold-Out ROC AUC Scores
auc_baseline = roc_auc_score(y_test, proba_baseline)
auc_enriched = roc_auc_score(y_test, proba_enriched)

# Print the final results
print("=== HOLD-OUT TEST RESULTS ===")
print(f"Baseline Model Hold-Out AUC: {auc_baseline:.4f}")
print(f"Enriched Model Hold-Out AUC: {auc_enriched:.4f}")
```

✓ 0.4s

```
=== HOLD-OUT TEST RESULTS ===
Baseline Model Hold-Out AUC: 0.5904
Enriched Model Hold-Out AUC: 0.6069
```


ROC Curve - Hold-Out Test Set
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Enriched (AUC = 0.61)
Baseline (AUC = 0.59)
Random Guessing

Data Summary and Implications

E. Summarize the implications of your data analysis by discussing the results of your data analysis in the context of the research question, including one limitation of your analysis. Within the context of your research question, recommend a course of action based on your results. Then propose two directions or approaches for future study of the dataset.

Since the corrected resampled t-test yielded a p-value less than 0.05, the null hypothesis is rejected in favor of the alternative hypothesis. This implies that the model with event-enriched features showed a statistically significant improvement in long-term forecasting for Seattle Airbnbs. As seen on the ROC-AUC graph, the enriched model has 2% more area under the curve, meaning it is better able to distinguish between classes (booked vs. un-booked).

One Limitation: A limitation of my analysis is that the lead time was likely too far out, resulting in fewer bookings to reliably train the models on. While the study pivoted to address this data sparsity, the lack of booking volume far in advance may have capped the performance of both models.

Course of Action Based on Results: Given these findings, hyper-local event data should be considered when forecasting long-term demand for Seattle-based rentals. Although a 2% improvement in the AUC score may seem small, it is statistically significant compared with using only baseline data. Immediate action in the form of further optimization of the model, hyperparameters, features, and methodology is needed, since a 0.61 AUC score is only slightly better than a coin flip at predicting bookings.

Future Study:

1) **Closer Lead Times**: Future research should look at how the models perform at closer intervals, such as 30, 60, or 90 days out. This would help identify the exact timeline at which event data becomes most accurate and determine whether the data sparsity issue naturally resolves as the lead time shortens.

2) **Feature Engineering/Optimization**: Further research into engineering/optimizing event variables should be considered. Feature importance should be analyzed, and alternative features should be considered, including but not limited to holidays, weather conditions, seasonality, and a broader range of events.

References

Climate pledge arena. (2023). In *Wikipedia*. https://en.wikipedia.org/wiki/Climate_Pledge_Arena

Discovery API 2.0. (2026). In *The Ticketmaster Developer Portal*. https://developer.ticketmaster.com/products-and-docs/apis/discovery-api/v2/

Elphick, D. (2024). Booking lead time: Guide for hotels | SiteMinder. In *SiteMinder*. https://www.siteminder.com/r/booking-lead-time/

Google. (n.d.). Classification: ROC and AUC. In Google for Developers. Retrieved August 14, 2026, from https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc#area_under_the_curve_auc

Haversine distance — Distancia 0.0.1 documentation. (2020). In Readthedocs.io. https://distancia.readthedocs.io/en/latest/Haversine.html

Inside Airbnb. (2024). Get the data. In *insideairbnb.com*. https://insideairbnb.com/get-the-data/

Kateryna. (2021, May 20). *Designing machine learning experiment and interpreting experimental results*. Medium. https://drogaieva.medium.com/designing-machine-learning-experiment-and-interpreting-experimental-results-cbf870caf68d

Katz, H., Savage, E., & Coles, P. (2025). Lead Times in flux: Analyzing airbnb booking dynamics during global upheavals (2018–2022). Annals of Tourism Research Empirical Insights, 6(2), 100185–100185. https://doi.org/10.1016/j.annale.2025.100185

Kiani, J. (2019). Using the corrected paired student's T-test for comparing machine learning models. In *Analytics Vidhya*. https://medium.com/analytics-vidhya/using-the-corrected-paired-students-t-test-for-comparing-the-performance-of-machine-learning-dc6529eaa97f

Lumen field. (2021). In Wikipedia. https://en.wikipedia.org/wiki/Lumen_Field

Nadeau, C., & Bengio, Y. (2003). Inference for the Generalization Error. *Machine Learning*, *52*(3), 239–281. https://doi.org/10.1023/a:1024068626366

Rezaei, N., & Jabbari, P. (2022). Random forests in R. *Elsevier eBooks*, 169–179. https://doi.org/10.1016/b978-0-12-822400-7.00001-4

Scikit-Learn. (n.d.). Sklearn.Ensemble.RandomForestClassifier — Scikit-learn 0.20.3 documentation. In *Scikit-learn.org*. Retrieved August 12, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Sugavasilakshmisahithi. (2025). Winsorization: A simple and effective way to handle outliers in your data. In *Medium*. Data Science Collective. https://medium.com/data-science-collective/winsorization-a-simple-and-effective-way-to-handle-outliers-in-your-data-e9635c3bc049

T-mobile park. (2025). In *Wikipedia*. https://en.wikipedia.org/wiki/T-Mobile_Park